

AbilitySystem — 뭘 자랑할까?

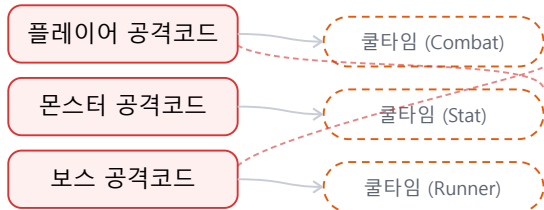
어려운 용어 없이, 그림으로. "무엇을 앞세우고 무엇을 낮출지."

자랑할 건 "어빌리티 시스템을 만들었다"가 아닙니다. 그건 남들도 다 말해요.
진짜 값나가는 건 "지저분하게 얹혀 있던 걸, 이유를 대며 깔끔하게 정리한 판단" — 이겁니다.

1 가장 큰 그림 — 지저분함을 하나로

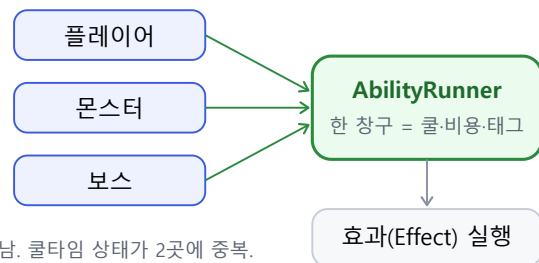
전에는 공격을 "누가 쓰느냐"에 따라 코드가 3벌이었다. 지금은 다 같은 창구 하나로.

🔴 전 (Before) — 발동 방식이 3벌, 쿨타임이 여기저기



→ 새 캐릭터·스킬마다 코드(클래스)가 계속 늘어남. 쿨타임 상태가 2곳에 중복.

🟢 후 (After) — 셋 다 같은 창구 하나로



→ 새 스킬 = 코드가 아니라 "데이터 조립". 쿨타임은 한 곳에서만 관리.

붉은 쪽(얍힘) → 초록 쪽(창구 하나). 이 "정리했다"가 이야기의 뼈대.

2 값나가는 것 4가지 — 쉬운 말로

면접관이 "오, 생각을 했네"라고 느끼는 지점들.



새 스킬 = 레고 조립

"방어력 깎기" 부품 하나를 숫자·방향만 바꿔 가드(내방어 ↑)·브레이커(적 방어 ↓)·버퍼(아군 공격 ↑) 셋에 재사용. 새 스킬 만들 때 코드를 안 짜고 데이터만 조합.

👉 "콘텐츠가 늘어도 프로그래머가 병목이 안 된다" = 기획자와 협업하는 실무 감각



정리했더니 코드가 사라짐

쿨타임을 "매 프레임 깎기"가 아니라 "끝나는 시각을 기억"하는 방식으로 바꾸니, 스왑한 캐릭터 쿨을 대신 돌려주던 코드 뭉치가 통째로 불필요해져 삭제.

👉 "좋은 데이터 모델이 코드를 지운다" = 설계로 문제를 없앤 이야기



안 만든 것을 설명할 수 있음

멀티플레이 동기화, 복잡한 스킬트리 같은 거창한 건 일부러 뺐다. "싱글게임이라 필요 없다"를 근거로 말할 수 있음.

👉 무작정 크게 만들지 않고 선을 긋는 판단 = 잘 안 나오는 강점

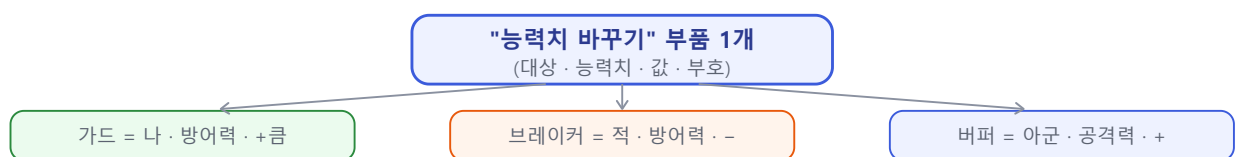


타격 순간에 정확히 발사

총알·효과가 애니메이션의 "던지는 프레임"에 맞춰 나가게 함. 쿨·공게이지도 누르는 순간 빠져 컷신 내내 도는 게 보임.

👉 "플레이 감각"을 신경 쓴 흔적 = 게임플레이 직무가 딱 원하는 것

* ①을 그림으로 — 부품 하나가 세 역할



코드는 하나, 결과는 셋 — 이게 "데이터 주도"의 진짜 증거.

3 숫자로 (말보다 강함)

"만들었다" 대신 "이만큼 줄였다"로 말하기.

3 → 1

공격 발동 창구
(플레이어·몹·보스 → 하나)

N → 0

새 스킬 만들 때
필요한 새 코드(클래스)

10 +

삭제한 캐릭터
전용 클래스

+ 쿨타임 관리 코드 뭉치(CharacterCombat, 매 프레임 돌리던 TickCooldowns 등) 삭제.

4 솔직한 약점 — 알고 있어야 함

아직 약한 곳

"빠르다·가볍다"를 숫자로 못 보여준다. "메모리 안 쓰게 만들었다"는 주장을 프로파일러(측정 도구)로 Before/After 한 장만 찍으면 이 약점이 메워집니다. 지금 최우선 보강 1순위.

5 그래서 한 문장으로 (헤드라인)

이렇게 말하세요

"플레이어·몬스터·보스가 각자 갖고 있던 공격 발동 코드와, 여기저기 흩어진 쿨타임을 — 데이터로 조립하는 단일 파이프라인으로 정리했습니다. 그 결과 새 스킬은 코드 없이 데이터만으로 만들고, 발동 경로는 3개에서 1개가 됐습니다."

→ 핵심은 "시스템 자랑"이 아니라 **"지저분한 걸 이유를 대며 정리한 판단 + 그 결과 숫자"**. 이 각도로 가면 강합니다.